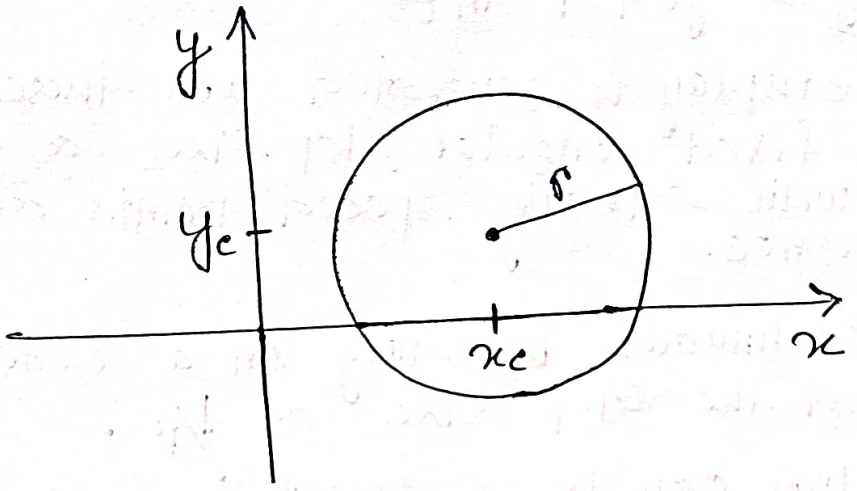
Circle generation algorithms :

A circle is defined as the set of points that are all at a given distance r from a center.



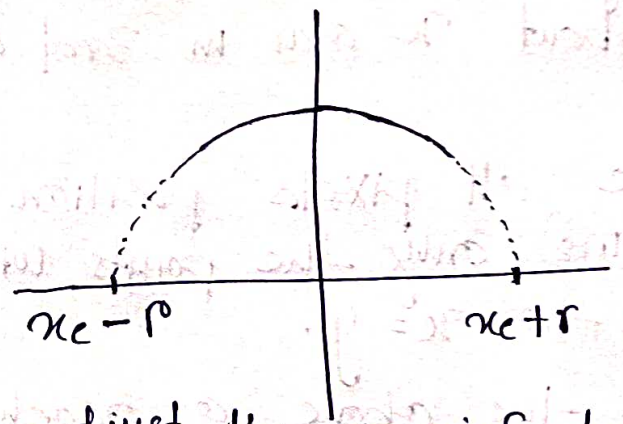
The equation of a circle is given as :

$$(x-x_c)^2 + (y-y_c)^2 = r^2$$

We can draw the circle by stepping along the x-axis unit steps from $x_c - r$ to $x_c + r$ and calculating the corresponding y-value at each position as :

$$y = y_c \pm \sqrt{r^2 - (x_c - x)^2}$$

But this process involves considerable computation at each step. Also the spacing between plotted pixel is not uniform.



We could adjust the spacing by interchanging x and y and stepping through y value.

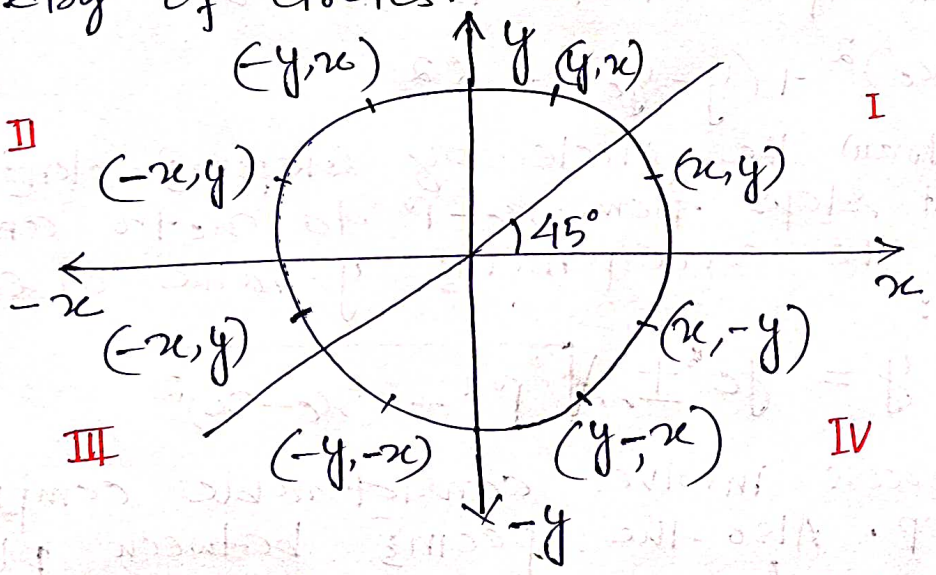
Another way to draw circle using polar coordinates r and θ.

$$x = x_c + r\cos\theta$$
$$y = y_c + r\sin\theta$$

When a display is generated with these equations using a fixed angular step size, a circle is plotted with equally spaced points along the circumference.

For a continuous boundary on a raster display we can set the step size at $1/r$.

Computation can be reduced by considering the symmetry of circles.



Symmetry about y axis in the 2nd quadrant and symmetry about x axis in 3rd and 4th quadrant.

We can generate all pixels position around a circle by calculating only the points within sector $x = 0$ ~~and $y = 0$~~ to $x = y$.

The problem with the above approach is that these involves multiplication, square root and trigonometric calculation.

Midpoint circle algorithm:

For a given radius r and screen center position (x_c, y_c) we can first set up our algorithm to calculate pixel positions around a circle path centered at the co-ordinate origin $(0, 0)$.

Then each calculated position (x, y) is moved to its proper screen position by adding x_c and y_c to x and y.

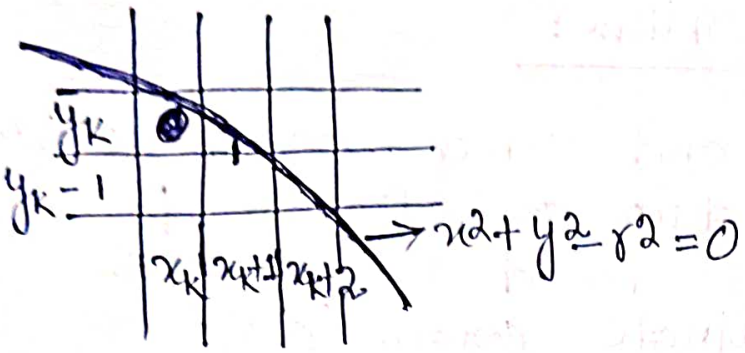
we can define circle function:

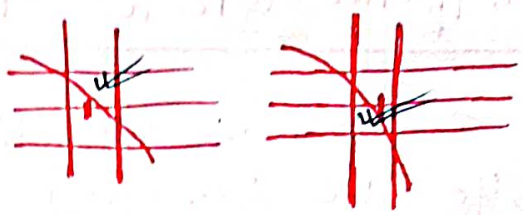
$$f_{circle}(x, y) = x^2 + y^2 - r^2$$

Any point (x, y) on the boundary of the circle with radius r satisfies the equation.

$$f_{circle}(x, y) = 0$$

$$f_{circle}(x, y) = \begin{cases} < 0 & \text{if } (x, y) \text{ is inside circle} \\ = 0 & \text{if } (x, y) \text{ is on the circle} \\ > 0 & \text{if } (x, y) \text{ is outside the circle boundary.} \end{cases}$$





* Assume that we just plotted the pixel at (x_K, y_K). we need to determine whether the pixel at position (x_K+1, y_K) or (x_K+1, y_K-1) is closer to the circle.

$$P_K = f_{circle}(x_K + 1, y_K - \tfrac{1}{2})$$
$$= (x_K + 1)^2 + (y_K - \tfrac{1}{2})^2 - r^2$$

* If $P_K < 0$, the midpoint is inside the circle and the pixel on scan line y_K is closer to the circle. else the midpoint is out side the circle and we will select the pixel on scan line $y_K - 1$.

$$P_{K+1} = f_{circle}(x_{K+1} + 1, y_{K+1} - \tfrac{1}{2})$$
$$= [(x_K + 1) + 1]^2 + (y_{K+1} - \tfrac{1}{2})^2 - r^2$$

$$P_{K+1} - P_K = [(x_K + 1) + 1]^2 + (y_{K+1} - \tfrac{1}{2})^2 - r^2 - (x_K + 1)^2 - (y_K - \tfrac{1}{2})^2 + r^2$$

$$= (x_K + 1)^2 + 1 + 2(x_K + 1) + y_{K+1}^2 + (\tfrac{1}{2})^2 - y_{K+1} - (x_K + 1)^2 - y_K^2 - (\tfrac{1}{2})^2 + y_K$$

$$P_{K+1} = P_K + 2(x_K + 1) + (y_{K+1}^2 - y_K^2) - (y_{K+1} - y_K) + 1$$

where y_{K+1} is either y_K or y_{K-1} depending on sign of P_K

If $P_k < 0$ then $(y_{k+1} = y_k)$

So $P_{k+1} = P_k + 2(x_k + 1) + 1$

$= P_k + 2(x_{k+1}) + 1$

else $P_{k+1} = 2x_{k+1} + 1 - 2y_{k+1}$ $(y_{k+1} = y_{k-1})$

$y_{k+1}^2 - y_k^2 - y_{k+1} + y_k + 1$

$\Rightarrow (y_k - 1)^2 - y_k^2 - y_{k-1} + y_k - 1$

$\Rightarrow (y_k - 1)^2 - y_k^2 - y_k + 1 + y_k - 1$

$\Rightarrow y_k^2 + 1 - 2y_k - y_k^2 - y_k + 1 + y_k - 1$

$\Rightarrow 1 - 2y_k$

$\Rightarrow -2y_k + 1$

$\Rightarrow \boxed{-2y_{k+1}}$

Evaluation of the terms $2x_{k+1}$ and $2y_{k+1}$ can also be done incrementally as:

$2x_{k+1} = 2x_k + 2$

$2y_{k+1} = 2y_k - 2$

The initial point is $(x_0, y_0) = (0, r)$

$P_0 = f_{circle}(1, r - \frac{1}{2})$

$\Rightarrow P_0 = 1 + (r - \frac{1}{2})^2 - r^2$

$\Rightarrow P_0 = \frac{5}{4} - r$

If the radius r is specified as an integer we can simply round P_0 to

$$\boxed{P_0 = 1 - r}$$

Algorithm :

1. Input radius r and circle center and obtain the first point on the circumference of a circle on the origin as : $(x_0, y_0) = (0, r)$
2. Calculate the initial value of the decision parameter as : $P_0 = \frac{5}{4} - r$
3. At each x_k position, starting at $k=0$, perform the following test. If $P_k < 0$, the next point along the circle centered on $(0,0)$ is (x_{k+1}, y_k) and
$$P_{k+1} = P_k + 2x_{k+1} + 1$$
Otherwise, the next point along the circle is (x_k+1, y_k-1) and
$$P_{k+1} = P_k + 2x_{k+1} + 1 - 2y_{k+1}$$
where $2x_{k+1} = 2x_k + 2$ and $2y_{k+1} = 2y_k - 2$
4. Determine symmetry points in the other seven octants.
5. Move each calculated pixel position (x, y) onto the circular path centered on (x_c, y_c) and plot the coordinate values :
$$x = x + x_c \quad , \quad y = y + y_c$$
6. Repeat step 3 through 5 until $x \geq y$.

Example: Given $r = 10$, center of circle $(0, 10)$
Determine positions of the circle in the 1st quadrant from $x = 0$ to $x = y$.

Soln

$P_0 = 1 - r = -9$

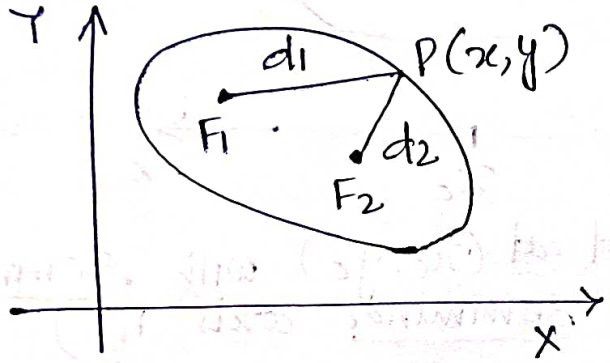
$2x_0 = 0$

$2y_0 = 20$

k	P_k	(x_{k+1}, y_{k+1})	$2x_{k+1}$	$2y_{k+1}$
0	-9	(1, 10)	2	20
1	-6	(2, 10)	4	20
2	-1	(3, 10)	6	20
3	6	(4, 9)	8	18
4	-3	(5, 9)	10	18
5	8	(6, 8)	12	16
6	5	(7, 7)	14	14

Ellipse Generation Algorithms:

An ellipse is defined as the set of points such that the sum of distances from two fixed positions is the same for all points.



The general equation of an ellipse can be stated as:

$$d_1 + d_2 = \text{Constant} \quad \text{——— (1)}$$

- where d_1 and d_2 are the distances to the two foci from any point $P(x, y)$ on the ellipse.

Expressing distances d_1 and d_2 in terms of the focal coordinates $F_1(x_1, y_1)$ and $F_2(x_2, y_2)$ we have:

$$\sqrt{(x-x_1)^2 + (y-y_1)^2} + \sqrt{(x-x_2)^2 + (y-y_2)^2} = \text{Constant} \quad \text{——— (2)}$$

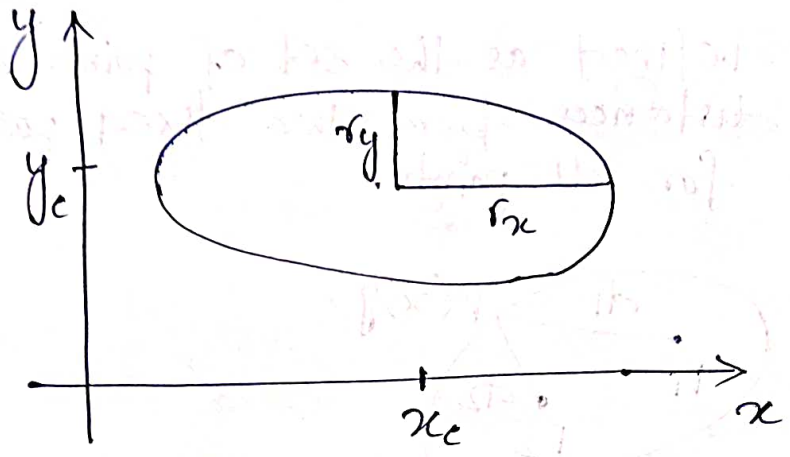
Simplifying the above expression we can write the general ellipse equation in the form:

$$Ax^2 + By^2 + Cxy + Dx + Ey + F = 0 \quad \text{——— (3)}$$

An ellipse can be specified in any arbitary orientation with the help of <u>two foci</u> and <u>a point on the ellipse boundary</u>.

With these above co-ordinates we can evaluate constants in eqn (2)

Then the coefficients in eqn (3) can be evaluated and used to generate pixels along the elliptical path.



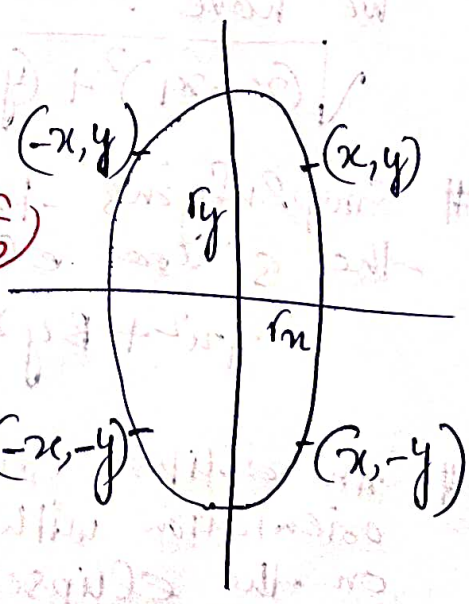
(Ellipse centered at (x_c, y_c) with semimajor axis r_x and semiminor axis r_y)

The equation of ellipse can be written in terms of the ellipse center coordinates and parameter r_x and r_y as:

$$\left(\frac{x-x_c}{r_x}\right)^2 + \left(\frac{y-y_c}{r_y}\right)^2 = 1 \quad \text{——(4)}$$

Using polar coordinates r an θ, we can also describe the ellipse in standard position with the parametric equations:

$$x = x_c + r_x \cos\theta$$
$$y = y_c + r_y \sin\theta \quad \text{——(5)}$$



An ellipse in standard position is symmetric between quandrants.

We must calculate pixel positions along the elliptical arc throughout one quadrant then we obtain positions in the remaining three quadrants by symmetry.

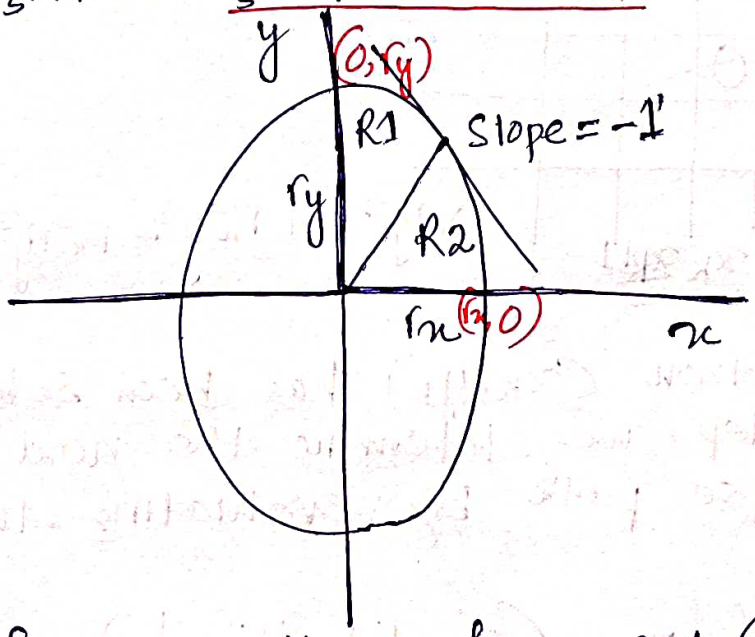
Midpoint Ellipse Algorithm :

Given parameters r_x, r_y and (x_c, y_c) we determine points (x, y) for an ellipse in standard position centered on the origin, and then we shift the points so the ellipse is centered at (x_c, y_c).

The midpoint ellipse method is applied throughout the first quadrant in two parts, according to the slope with $r_x < r_y$.

We process this quadrant by taking unit steps in the x direction where the slope of the curve has magnitude less than 1.

Taking units steps in y direction where the slope has magnitude greater than 1.



We define an ellipse fom eqn (4) with $(x_c, y_c) = (0,0)$ as

$$f_{ellipse} = r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2 \quad \text{—— (6)}$$

which has the following property :

$$f_{ellipse}(x,y) \begin{cases} < 0 & \text{if } (x,y) \text{ inside the ellipse} \\ = 0 & \text{if } (x,y) \text{ on the ellipse} \\ > 0 & \text{if } (x,y) \text{ outside the ellipse} \end{cases} \quad \text{—— (7)}$$

The ellipse function $f_{ellipse}(x, y)$ serves as the decision parameter in the midpoint algorithm.

The ellipse slope is calculated from eqn (6) as

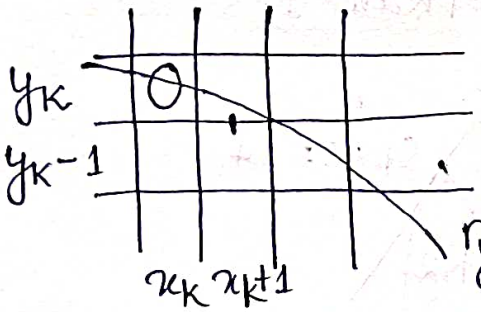
$$\frac{dy}{dx} = -\frac{2r_y^2 x}{2r_x^2 y} \quad \text{—— (8)}$$

At the boundary between region 1 and region 2

$$\frac{dy}{dx} = -1 \quad \text{and} \quad 2r_y^2 x = 2r_x^2 y$$

Therefore, we move out of region 1 whenever

$$2r_y^2 x \geq 2r_x^2 y \quad \text{—— (9)}$$



Assuming position (x_K, y_K) has been selected at the previous step, we determine the next position along the ellipse path by evaluating the decision parameter :

$$p1_K = f_{ellipse}(x_K + 1, y_K - \tfrac{1}{2})$$
$$= r_y^2 (x_K + 1)^2 + r_x^2 (y_K - \tfrac{1}{2})^2 - r_x^2 r_y^2$$

—— (10)

If $p1_K < 0$, the midpoint is inside the ellipse and the pixel position on scan line y_K is closer to the ellipse boundary.

Otherwise, the mid position is outside the ellipse so we select the pixel on scan line $y_K - 1$

At the next sampling position ($x_{k+1}+1 = x_k+2$) the decision parameter for region 1 is evaluated as:

$$P1_{k+1} = f_{ellipse}(x_{k+1}+1, y_{k+1}-\tfrac{1}{2})$$
$$= r_y^2[(x_k+1)+1]^2 + r_x^2(y_{k+1}-\tfrac{1}{2})^2 - r_x^2 r_y^2$$

or

$$P1_{k+1} = P1_k + 2r_y^2(x_k+1) + r_y^2 + r_x^2[(y_{k+1}-\tfrac{1}{2})^2 - (y_k-\tfrac{1}{2})^2] \quad \text{——— (11)}$$

where y_{k+1} is either y_k or $y_k - i$ depending on the sign of $P1_k$.

decision parameters are incremented by the following amounts.

$$\text{increments} = \begin{cases} 2r_y^2 x_{k+1} + r_y^2 & \text{if } P1_k < 0 \\ 2r_y^2 x_{k+1} + r_y^2 - 2r_x^2 y_{k+1} & \text{if } P1_k \geq 0 \end{cases}$$

At the initial position $(0, r_y)$ the two terms evaluate to

$$2r_y^2 x = 0 \quad \text{——— (12)}$$
$$2r_x^2 y = 2r_x^2 r_y \quad \text{——— (13)}$$

In region 1, the initial value of the decision parameter is obtained by evaluating the ellipse function at the start position $(x_0, y_0) = (0, r_y)$

$$P1_0 = f_{ellipse}(1, r_y - \tfrac{1}{2})$$
$$= r_y^2 + r_x^2(r_y - \tfrac{1}{2})^2 - r_x^2 r_y^2$$

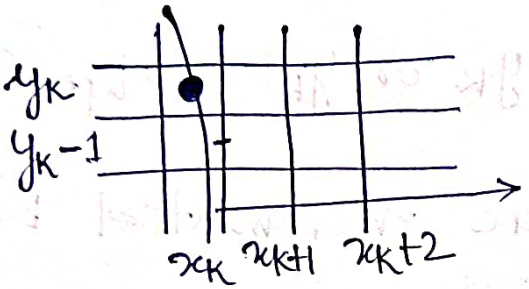
$$\Rightarrow P1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2 \quad \text{——— (14)}$$

Over region 2, we sample at unit steps in the negative y direction and the midpoint is now taken between horizontal pixel at each step.

Now the decision parameter is evaluated as:

$$P2_K = f_{ellipse}(x_K + \tfrac{1}{2}, y_K - 1)$$

$$= r_y^2 (x_K + \tfrac{1}{2})^2 + r_x^2 (y_K - 1)^2 - r_x^2 r_y^2 \quad \text{---------- (15)}$$



$$r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2 = 0$$

If $P2_K > 0$, the midposition is outside the ellipse boundary and we select the pixel at x_K.

If $P2_K \le 0$, the midpoint is inside or on the ellipse boundary, and we select pixel position x_{K+1}

Now the decision parameter can evaluate as:

$$P2_{K+1} = f_{ellipse}(x_{K+1} + \tfrac{1}{2}, y_{K+1} - 1)$$

$$= r_y^2 (x_{K+1} + \tfrac{1}{2})^2 + r_x^2 [(y_K - 1) - 1]^2 - r_x^2 r_y^2$$

$$P2_{K+1} = P2_K - 2r_x^2 (y_K - 1) + r_x^2 + r_y^2 [(x_{K+1} + \tfrac{1}{2})^2 - (x_K + \tfrac{1}{2})^2]$$

with x_{K+1} set either to x_K or x_{K+1}, depending on the sign of $P2_K$.

When we enter region 2, the initial position (x_0, y_0) is taken as the last position selected in region 1. and the initial decision parameter in region 2 is

$$P2_0 = f_{ellipse}(x_0 + \tfrac{1}{2}, y_0 - 1)$$

$$= r_y^2 (x_0 + \tfrac{1}{2})^2 + r_x^2 (y_0 - 1)^2 - r_x^2 r_y^2$$

Algorithm:

1. Input r_x, r_y and ellipse center (x_c, y_c) and obtain the first point on an ellipse centered on the origin as:

$$(x_0, y_0) = (0, r_y)$$

2. Calculate the initial value of the decision parameter in region 1 as:

$$P1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$$

3. At each x_k position in region 1, starting at $k=0$, perform the following test:

If $P1_k < 0$, the next point along the ellipse centered on $(0,0)$ is (x_{k+1}, y_k) and

$$P1_{k+1} = P1_k + 2r_y^2 x_{k+1} + r_y^2$$

Otherwise, the next point along the circle is (x_k+1, y_k-1) and

$$P1_{k+1} = P1_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$$

with $2r_y^2 x_{k+1} = 2r_y^2 x_k + 2r_y^2$

$2r_x^2 y_{k+1} = 2r_x^2 y_k - 2r_x^2$

4. Calculate the initial value of the decision parameter in region 2 using the last point (x_0, y_0) calculated in region 1 as:

$$P2_0 = r_y^2 (x_0 + \tfrac{1}{2})^2 + r_x^2 (y_0 - 1)^2 - r_x^2 r_y^2$$

5. At each y_k position in region 2, starting at $k=0$, perform the following test:

If $P2_k > 0$, the next point along the ellipse centered on $(0,0)$ is $(x_k, y_k - 1)$ and

$$P2_{k+1} = P2_k - 2r_x^2 y_{k+1} + r_x^2$$

Otherwise the next point along the circle is (x_k+1, y_k-1) and

$$p2_{k+1} = p2_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$$

using the same incremental calculation for x and y as in region 1.

6. Determine symmetry points in the other three quadrants.
7. Move each calculated pixel position (x, y) onto the elliptical path centered on (x_c, y_c) and plot the co-ordinate values.

$$x = x + x_c, \quad y = y + y_c$$

8. Repeat the steps for region 1 until $2r_y^2 x \geq 2r_x^2 y$.

Example:

Given $r_x = 8$, $r_y = 6$

Soln $2r_y^2 x = 0$, $2r_x^2 y = 2r_x^2 r_y$

for region 1: Initial point is $(0, 6)$

$$p1_0 = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2 = -332$$

k	$p1_k$	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	-332	(1, 6)	72	768
1	-224	(2, 6)	144	768
2	-44	(3, 6)	216	768
3	208	(4, 5)	288	640
4	-108	(5, 5)	360	640
5	288	(6, 4)	432	512
6	244	(7, 3)	504	384

for region 2: the initial point is $(x_0, y_0) = (7, 3)$

$$p2_0 = f(7 + \tfrac{1}{2}, 2) = -151$$

k	$p2_k$	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	-151	(8, 2)	576	256
1	233	(8, 1)	576	128
2	745	(8, 0)	—	—